Portfolio & Goal Tracker — Build Specification

Nov 10, 2026

Overview

Build a personal investment tracker for stocks and mutual funds. The user logs each buy (instrument, quantity, price, date); the app pulls live/daily prices and NAVs, computes P/L% over fixed lookback windows, and runs SIP/goal math off the portfolio's trailing 1-year return. Single user, no multi-tenant auth needed unless specified otherwise.

Holdings input & data model

Each holding entry (one row per buy transaction, so averaging and lot-level P/L both work):

Field	Type	Notes
instrument_type	enum	`stock` or `mutual_fund`
symbol / scheme_code	text	exchange ticker (e.g. `RELIANCE.NS`) or AMFI scheme code for MF
name	text	display name, auto-filled from lookup
quantity	number	shares, or units (MF units can be fractional)
buy_price	number	price per share/unit at purchase, INR
buy_date	date	transaction date
exchange	enum	NSE/BSE for stocks, N/A for MF

Derived per holding: invested_value = quantity × buy_price; current_value = quantity × latest_price; absolute_pnl = current_value − invested_value. Portfolio-level figures are the sum across all holdings. Multiple buys of the same instrument stay as separate lots internally but roll up under one instrument in the UI.

Selling: a sell is its own transaction, not a deletion of the buy row — otherwise you lose the realized P/L history. Add a `transaction_type` field (`buy` or `sell`) to the table above. A sell record needs: instrument, quantity sold, sell_price, sell_date. On a sell:

- Reduce the open quantity against the oldest buy lot(s) first (FIFO) unless you want to let the user pick which lot to sell against.
- realized_pnl for that sale = (sell_price − buy_price of the matched lot) × quantity sold.
- The matched lot's remaining quantity (if any) stays open and keeps contributing to current_value and unrealized P/L; a fully-sold lot drops out of current holdings but stays in history.

Portfolio P/L should then split into **unrealized** (open lots, what the P/L% by period in the next section covers) and **realized** (closed/partial sales, running total) — showing only one of the two understates what the user actually made or lost.

Price & NAV updates

Google doesn't publish a public Google Finance API — `GOOGLEFINANCE()` only works inside Sheets. Two realistic paths:

1. **Mutual fund NAV — mfapi.in**: `https://api.mfapi.in/mf/{schemeCode}` for a scheme's full NAV history (and `/mf/search?q=...` to find a scheme code by name). Free, no key, community-run, wraps AMFI's own daily data directly.
2. **Stocks (NSE/BSE) — Yahoo Finance's JSON endpoint**: ticker with `.NS` (NSE) or `.BO` (BSE) suffix, e.g. `RELIANCE.NS`, `500325.BO`. Free, no key; gives live/~15-min-delayed price, previous close, and 52-week high/low. It's unofficial and undocumented by Yahoo (no SLA, can rate-limit or shift shape without notice) — fine for a casual personal tracker, just cache the last good price and fall back to it if a call fails so one hiccup doesn't blank the dashboard.

For a casual personal tracker, free-and-keyless beats paid-and-guaranteed. Build a scheduled job (cron / cloud function) that runs daily after both sources publish, refreshes `latest_price` on every holding, and snapshots portfolio total value with a timestamp — that snapshot history is what the P/L-by-period calculations below read from. Stock prices can refresh more often during market hours if the Yahoo endpoint holds up; mutual fund NAV is a once-daily batch regardless.

P/L% by period

Windows to show: 7 days, 30 days, 3 months, 6 months, 1 year, 5 years, and overall (since first buy).

For each window, P/L% = (value_now − value_at_window_start) / value_at_window_start × 100, where value_at_window_start is the portfolio's total value on that past date, taken from the daily snapshot history (section above) — not a theoretical back-calculation. If a holding didn't exist yet at the window's start date (bought more recently than the window), the window's starting value for that instrument is its invested_value on its own buy date, so new additions don't distort period returns for holdings that predate them. "Overall" P/L% uses total invested_value vs total current_value across all lots, regardless of date.

Display both the blended portfolio % and a per-instrument breakdown, since a single portfolio number can hide a big winner offsetting a big loser.

Goal Mode A — target amount + target time → required monthly SIP

Inputs: target_amount, target_date (or months_remaining). Assumed annual return r = the portfolio's trailing 1-year P/L%, converted to a monthly rate for the SIP formula. Solve the standard future-value-of-SIP equation for the monthly installment (formula in the Core formulas section). Output: required_monthly_sip.

Caveat to show the user: this assumes the next N years repeat last year's return, which for a single trailing year is a noisy estimate — worth flagging in the UI, not silently assumed.

Goal Mode B — target amount + monthly SIP → time to reach

Inputs: target_amount, monthly_sip. Same trailing-1-year return r. Solve the FV-of-SIP equation for N (months to target) instead of for the installment — formula in Core formulas. Output: projected_target_date. If r is zero or negative, say so explicitly rather than returning an undefined or infinite timeline.

Monthly goal recalculation

On a monthly schedule (cron), for every active goal:

1. Recompute trailing 1-year portfolio P/L% as of that month.
2. Re-run whichever mode created the goal, with the updated r and the amount already accumulated toward the target so far (not just the original target from scratch).
3. Mode A goals: output an adjusted_monthly_sip (how much the installment must change to still hit the original target_date).
4. Mode B goals: output an adjusted_target_date (new ETA at the current installment and current return).

5. Store each month's recalculation so the user can see how the required SIP or ETA has drifted over time, not just the latest number.

Core formulas

Monthly rate from the trailing annual return r (as a decimal, e.g. 0.18 for 18%):

```
i = (1+r)^{1/12} - 1
```

Future value of a monthly SIP of amount P for N months at monthly rate i:

```
FV = P \times \frac{(1+i)^N - 1}{i} \times (1+i)
```

Mode A — solve for P given target FV and N:

```
P = \frac{FV \times i}{\big[(1+i)^N - 1\big](1+i)}
```

Mode B — solve for N given target FV and fixed P (no closed form; solve numerically, e.g. binary search or Newton's method, since N sits inside both an exponent and the rest of the expression):

```
FV - P \times \frac{(1+i)^N - 1}{i} \times (1+i) = 0
```

If the recalculation already has an amount A accumulated, replace FV with (FV − A × (1+i)^{N}) in the solve, so past contributions compound forward rather than being double-counted.

Suggested stack & build notes

- Backend: Python (FastAPI) is a natural fit given existing Python comfort — one scheduled job handles the daily price/NAV pull and snapshotting, a second handles monthly goal recalculation.
- DB: Postgres (or SQLite for a solo v0) — tables: `holdings` (one row per buy), `price_snapshots` (daily, per instrument), `goals`, `goal_recalc_history`.
- Frontend: a simple dashboard — portfolio total + per-period P/L%, a holdings table, a goals section with the two creation modes and a history chart of adjusted SIP/ETA over time.
- Auth: skip unless this needs to be reachable from outside a trusted device — a single-user local/self-hosted app doesn't need it.
- Build order: holdings CRUD → daily price/NAV job + snapshot table → period P/L% → goal Mode A/B calculators → monthly recalc job + history view.